

CATAM Additional Projects

1.3 Parabolic Partial Differential Equations

2 Analytic Solution

Question 1

(i) Let

$$\begin{aligned}\theta(x, t) &= f(t)(1 - x) + \phi(x, t) \\ &= t(1 - t)(1 - x) + \phi(x, t)\end{aligned}$$

Then,

$$\begin{aligned}\theta_t &= (1 - 2t)(1 - x) + \phi_t(x, t) \\ \theta_{xx} &= \phi_{xx}(x, t).\end{aligned}$$

Substituting this into the diffusion equation we find the governing equation for ϕ :

$$\phi_t = \phi_{xx} + (2t - 1)(1 - x),$$

with boundary conditions for $0 \leq t < \infty$:

$$\begin{aligned}\phi(0, t) &= 0 \\ \phi(1, t) &= 0,\end{aligned}$$

and initial conditions for $0 \leq x \leq 1$:

$$\begin{aligned}\phi(x, t) &= -t(1 - t)(1 - x) \text{ for } t \leq 0 \\ \Rightarrow \phi(x, 0) &= 0.\end{aligned}$$

In the absence of the forcing term on the right-hand side, the governing equation for ϕ along with the boundary conditions can be solved via separation of variables to derive the Fourier sine series solution:

$$\phi = \sum_{n=1}^{\infty} A_n e^{-(n\pi)^2 t} \sin(n\pi x),$$

where A_n are specified by the initial conditions. We note that the forcing term itself has the Fourier sine series expansion for $0 < x < 1$:

$$\begin{aligned}(2t - 1)(1 - x) &= (2t - 1) \sum_{n=1}^{\infty} \frac{2}{n\pi} \sin(n\pi x) \\ &= \sum_{n=1}^{\infty} \frac{4t - 2}{n\pi} \sin(n\pi x).\end{aligned}$$

With both of these facts in mind we propose a solution to the forced equation of the form

$$\phi = \sum_{n=1}^{\infty} g_n(t) \sin(n\pi x).$$

Substituting this into the governing equation for ϕ we obtain an equation for g_n :

$$g'_n(t) = -(n\pi)^2 g_n(t) + \frac{4t-2}{n\pi}$$

which has solution

$$g_n(t) = \frac{4t}{(n\pi)^3} - \frac{2}{(n\pi)^3} - \frac{4}{(n\pi)^5} + A_n e^{-(n\pi)^2 t}.$$

Now imposing the initial conditions $\phi(x, 0) = 0 \Rightarrow g_n(0) = 0$, we have $A_n = \frac{4}{(n\pi)^5} - \frac{2}{(n\pi)^3}$ and hence obtain the full Fourier sine series solution for ϕ :

$$\phi(x, t) = \sum_{n=1}^{\infty} \left[\frac{4t}{(n\pi)^3} - \left(\frac{2}{(n\pi)^3} + \frac{4}{(n\pi)^5} \right) (1 - e^{-(n\pi)^2 t}) \right] \sin(n\pi x).$$

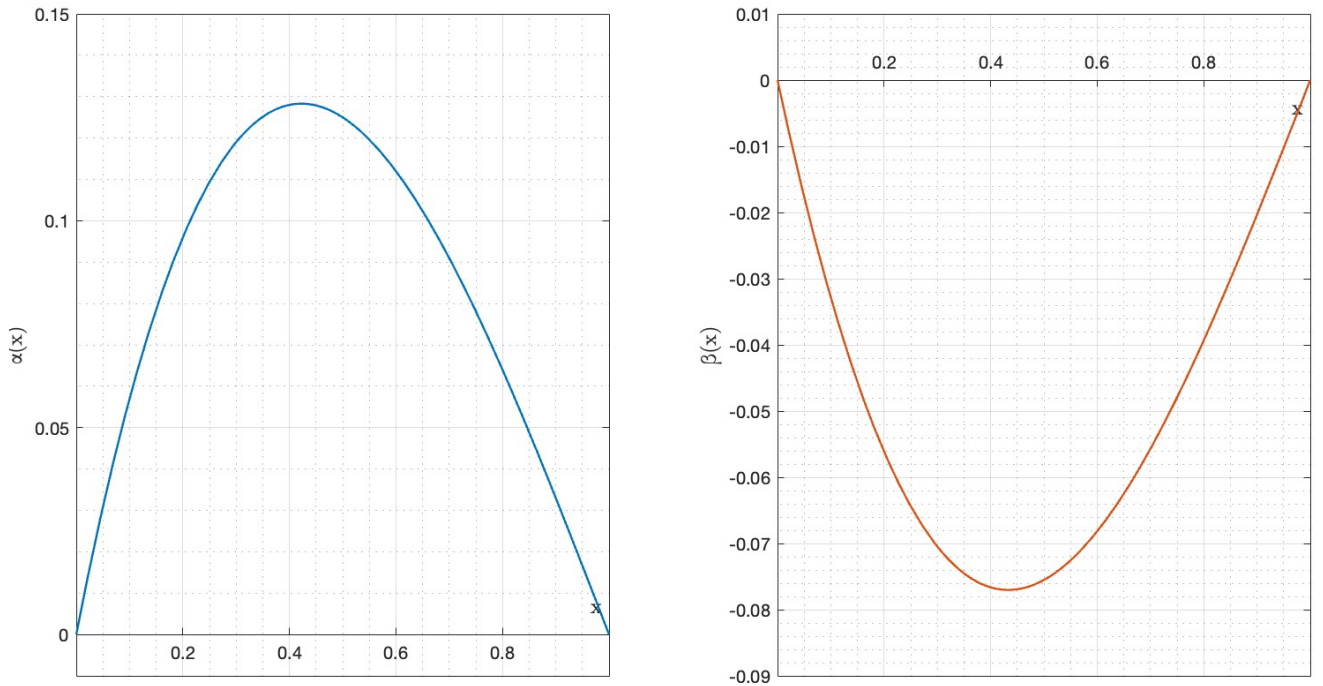
(ii) From the above expression for ϕ , we have

$$\begin{aligned} \phi &\rightarrow \sum_{n=1}^{\infty} \left(\frac{4t}{(n\pi)^3} - \frac{2}{(n\pi)^3} - \frac{4}{(n\pi)^5} \right) \sin(n\pi x) \quad \text{as } t \rightarrow \infty \\ &= \left(\sum_{n=1}^{\infty} \frac{4}{(n\pi)^3} \sin(n\pi x) \right) t - \sum_{n=1}^{\infty} \left(\frac{2}{(n\pi)^3} + \frac{4}{(n\pi)^5} \right) \sin(n\pi x) \end{aligned}$$

i.e.

$$\alpha(x) = \sum_{n=1}^{\infty} \frac{4}{(n\pi)^3} \sin(n\pi x) \quad \text{and} \quad \beta(x) = - \sum_{n=1}^{\infty} \left(\frac{2}{(n\pi)^3} + \frac{4}{(n\pi)^5} \right) \sin(n\pi x).$$

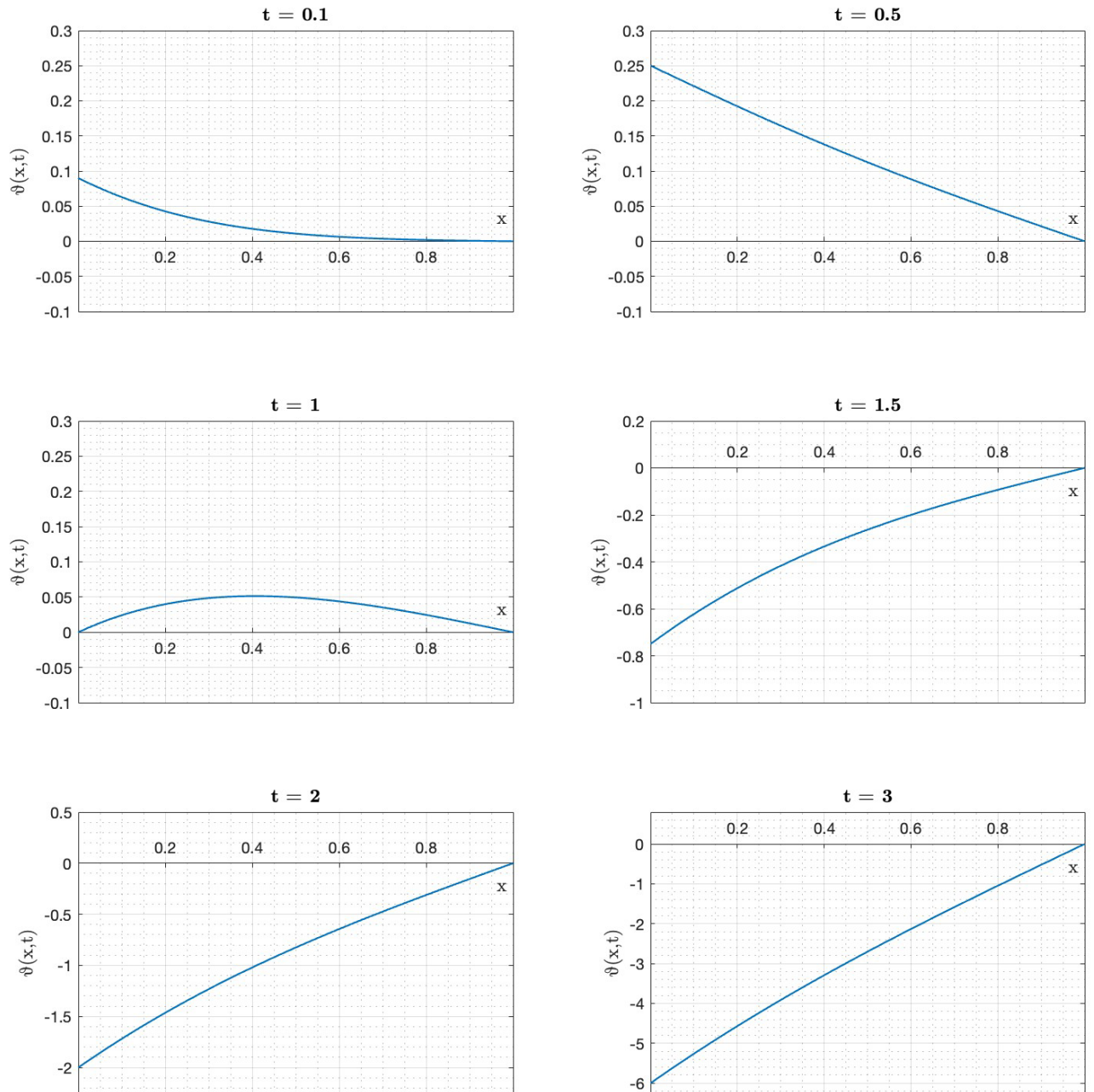
Figure 1.1 Plots of $\alpha(x)$ and $\beta(x)$ both found by summing the first 100 terms



(iii) A program computing the analytic solution by summing a finite number of terms is listed on page 18.

(iv) Figure 1.2 illustrates the evolution of the analytic solution from $t = 0.1$ to $t = 3$.

Figure 1.2 *Plots of $\theta(x)$ at various times, each found by summing the first 100 terms*



(v) It was found that summing more than 100 terms of the series solution for θ offered no improvement in graphical accuracy. Hence, in the interest of computational time, Figure 1.2 as well as all further plots of the analytical solution for θ were found by summing the first 100 terms. For later investigations into numerical errors, more accurate analytical values for θ were required. It was found that summing more than 20,000 terms only offered improvements on the order of 10^{-15} . This is close to the limit of computer precision at the scale we are working at and beyond the magnitude of the numerical errors we will be considering. Hence for such error investigations the analytical solution for θ was found by summing the first 20,000 terms.

(vi) As the temperature varies at $x = 0$ we expect the heat to diffuse through the bar with the distribution along the bar varying smoothly. At both early and late times, we see a temperature distribution that is curved and slightly steeper at $x = 0$ (see $t = 0.1$ & 0.5 and $t = 2$ & 3 in Figure 1.2). This is because the temperature at $x = 0$ is continuously increasing/decreasing at a slightly greater rate than diffusion can account for and hence the temperature distribution is steeper at $x = 0$. When the temperature at $x = 0$ decreases down to zero, the initial heating of the bar has left residual heat within the interior of the bar which we can see at $t = 1$ in Figure 1.2.

3 Numerical Methods

Question 2

(i) Programs implementing each of the numerical schemes are listed on pages 19 – 20. Figures 2.1 – 2.3 illustrate the numerical solutions for $\theta(x, t)$ found using schemes (i) – (iii) with $N = 5$, $\nu = \frac{1}{2}$ and $\rho = \frac{1}{2}$. We immediately notice the instability of scheme (ii) which is in fact unconditionally unstable (Ames, W.F. pp. 55) and henceforth it shall not be considered.

Figure 2.1 *Plots of the numerical solution for $\theta(x)$ using scheme (i)*

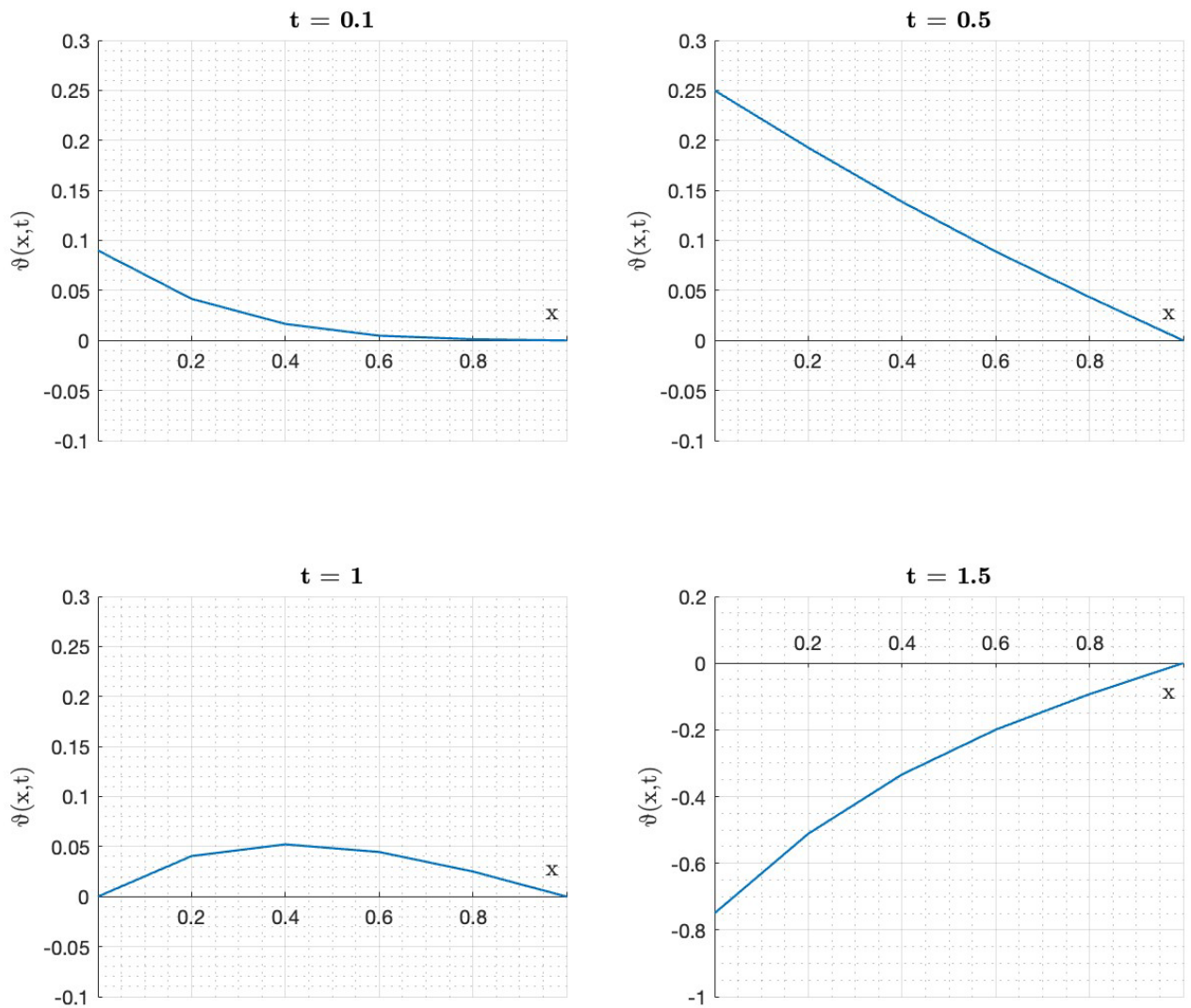


Figure 2.2 *Plots of the numerical solution for $\theta(x)$ using scheme (iii)*

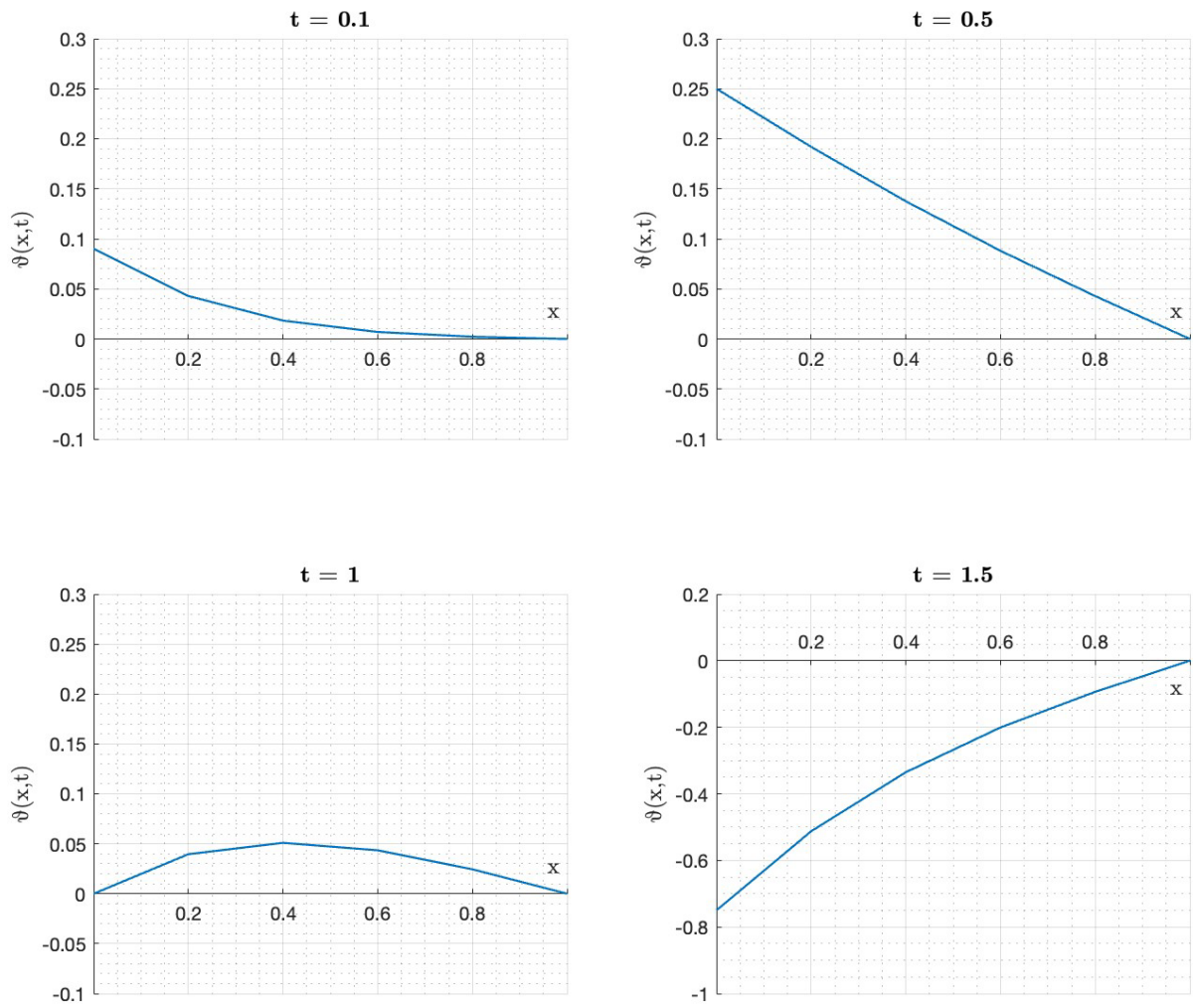


Figure 2.3 *Plots of the numerical solution for $\theta(x)$ using scheme (ii)*

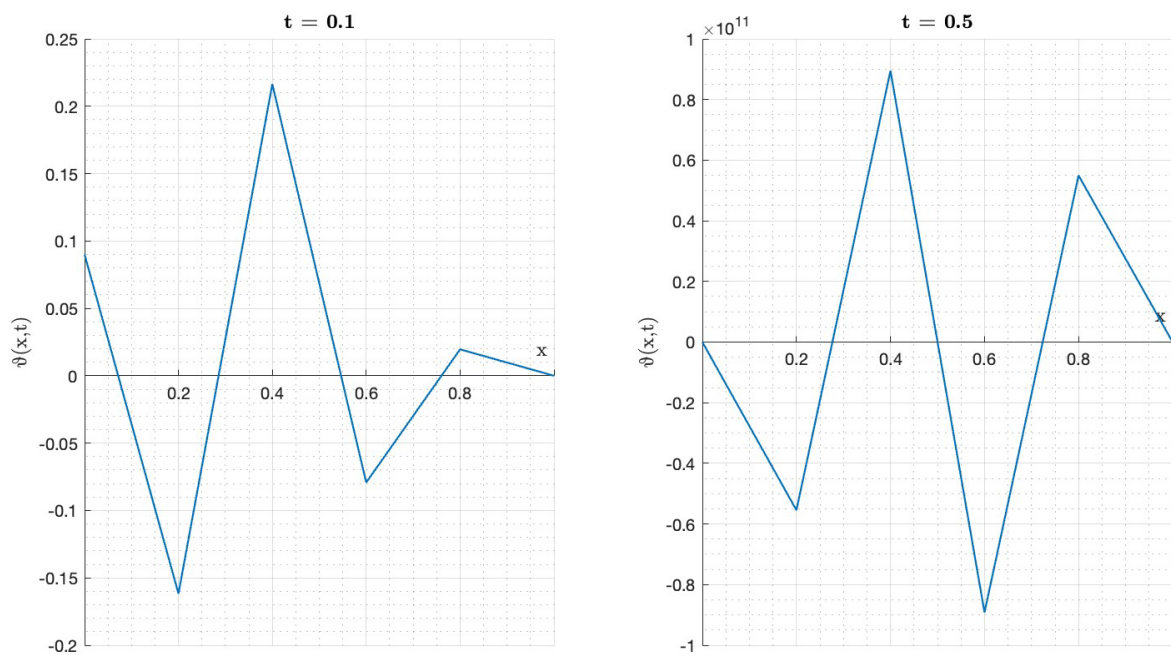
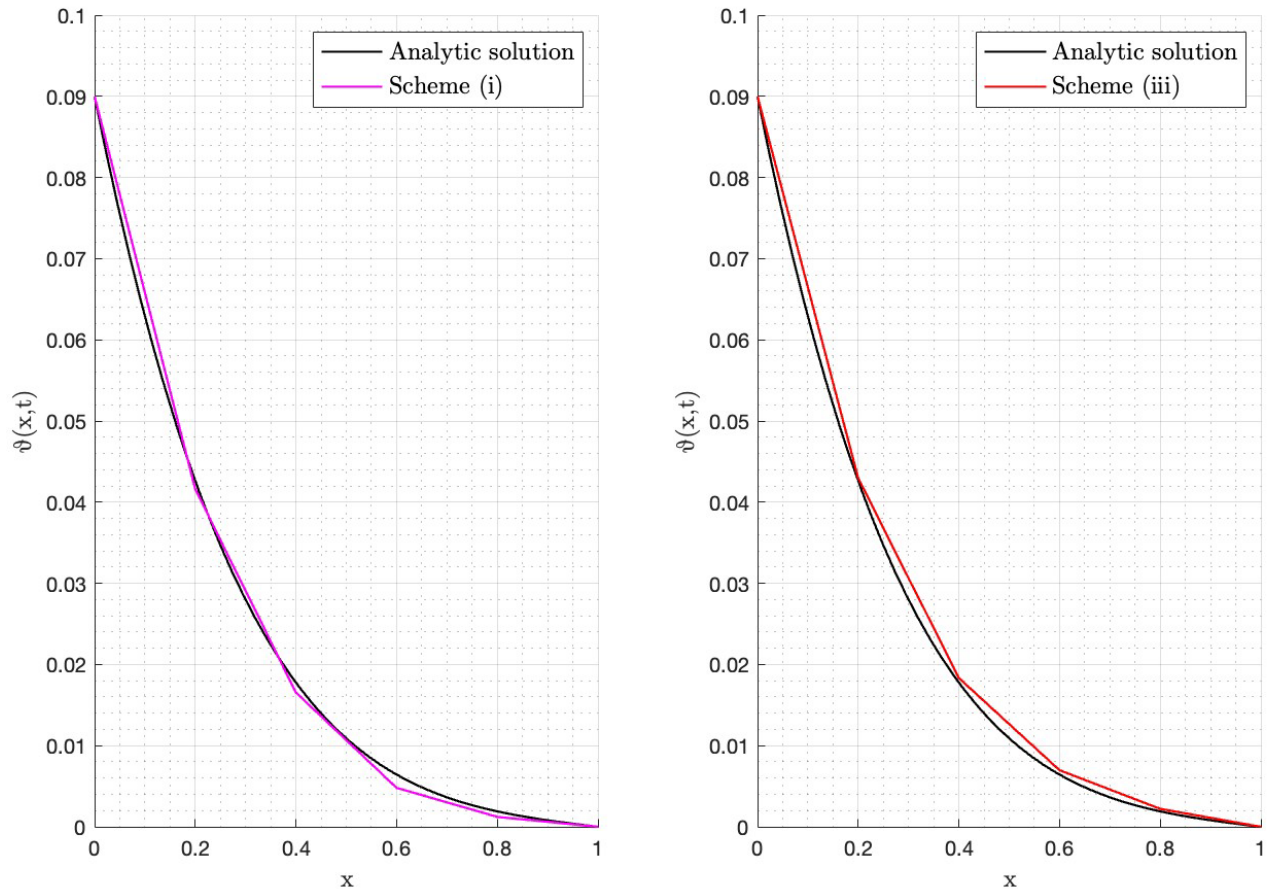


Table 2.1 Analytical & numerical solutions for $\theta(x, 0.1)$ with the numerical error

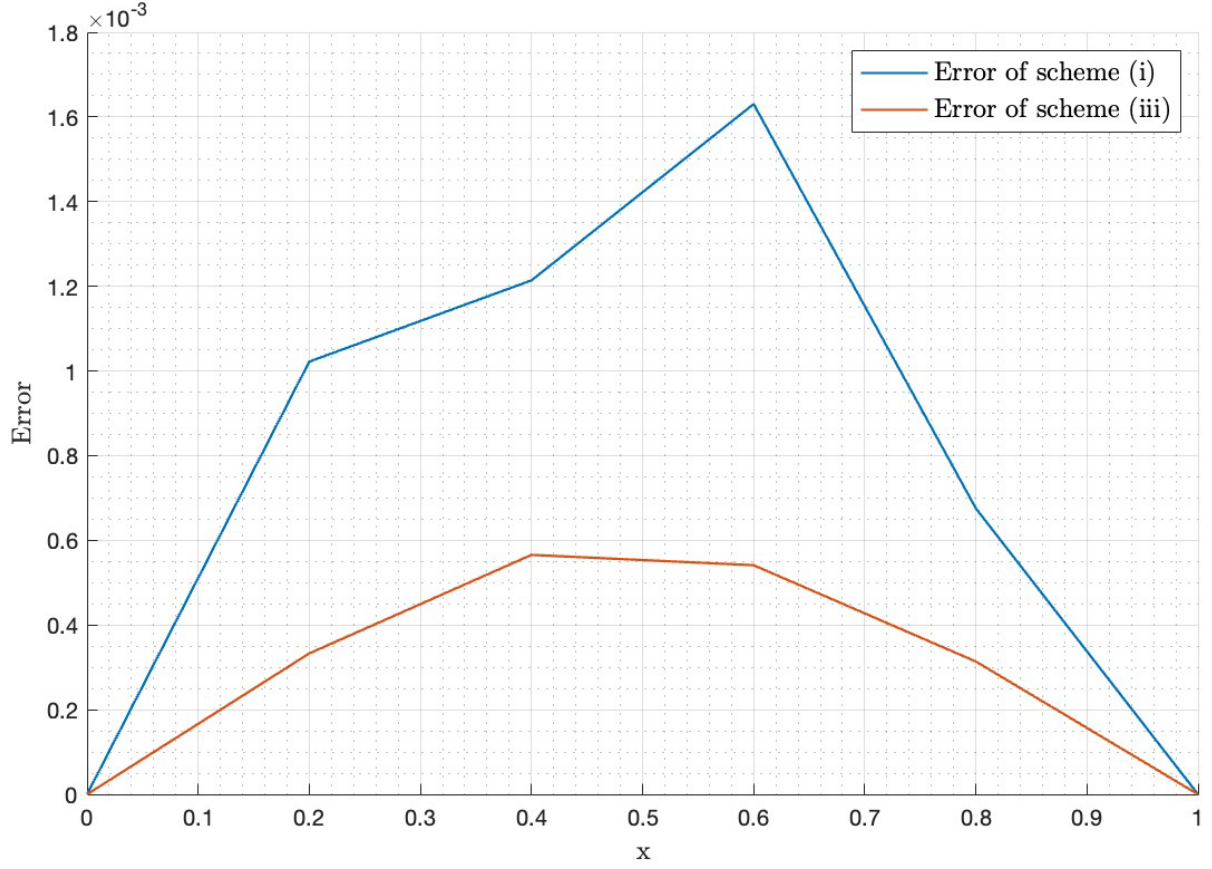
x	Analytic solution	Scheme (i)	Scheme (iii)	Scheme (i) error	Scheme (iii) error
0.2	0.04262212	0.0416	0.04295496	0.00102212	0.00033284
0.4	0.01776396	0.01655	0.01832951	0.00121396	0.00056556
0.6	0.00643046	0.0048	0.00697162	0.00163046	0.00054116
0.8	0.00190052	0.001225	0.00221426	0.00067552	0.00031374

Figure 2.4 Analytical & numerical solutions for $\theta(x, 0.1)$ with $N = 5, v = \frac{1}{2}$ & $\rho = \frac{1}{2}$



In Figures 2.1 & 2.2 the solutions found with schemes (i) & (iii) appear very similar, however differences between the two become visible once we look more closely at $t = 0.1$ in Figure 2.4. Table 2.1 records the analytic solution $\theta(n\delta x, m\delta t)$, the numerical solution θ_n^m for schemes (i) & (iii) as well as the error $\epsilon = |\theta_n^m - \theta(n\delta x, m\delta t)|$. Figure 2.5 plots this error and indicates that scheme (iii) provides a slightly more accurate solution with these parameters.

Figure 2.5 Numerical error for $\theta(x, 0.1)$ with $N = 5, \nu = \frac{1}{2}$ & $\rho = \frac{1}{2}$



(ii) & (iii) We must first note that for both schemes (i) & (iii) there exist conditions on the parameters to ensure *stability* and *convergence*. Naturally, we wish to only consider parameter values for which our numerical schemes are stable, however we must also concern ourselves with their convergence. The conditions regarding convergence are important for both schemes (i) & (iii) since it is possible to approximate a parabolic PDE by a finite-difference scheme which is stable and yet converges (as $\delta t \rightarrow 0$) to the solution of a *different* PDE (Smith, G.D. pp. 40). Hence, we must work with parameter values for which our numerical schemes converge to solutions of the PDE which they are employed to solve.

Let us first focus our attention on scheme (i). This scheme is stable and convergent for $\nu \leq \frac{1}{2}$ (Smith, G.D. *Example 2.8* pp. 54-55) and hence our subsequent investigations into scheme (i) consider only $\nu \leq \frac{1}{2}$. Indeed, brief investigation confirmed this result, with the drastic divergence of the scheme for values $\nu > \frac{1}{2}$. Taking $\delta t = \nu \delta x^2$ scheme (i) has local truncation error

$$\eta = \mathcal{O}(\delta x^2)$$

(Iserles, A. pp. 351). This suggests that the error $\epsilon = |\theta_n^m - \theta(n\delta x, m\delta t)|$ obeys:

$$\epsilon \propto \delta x^2$$

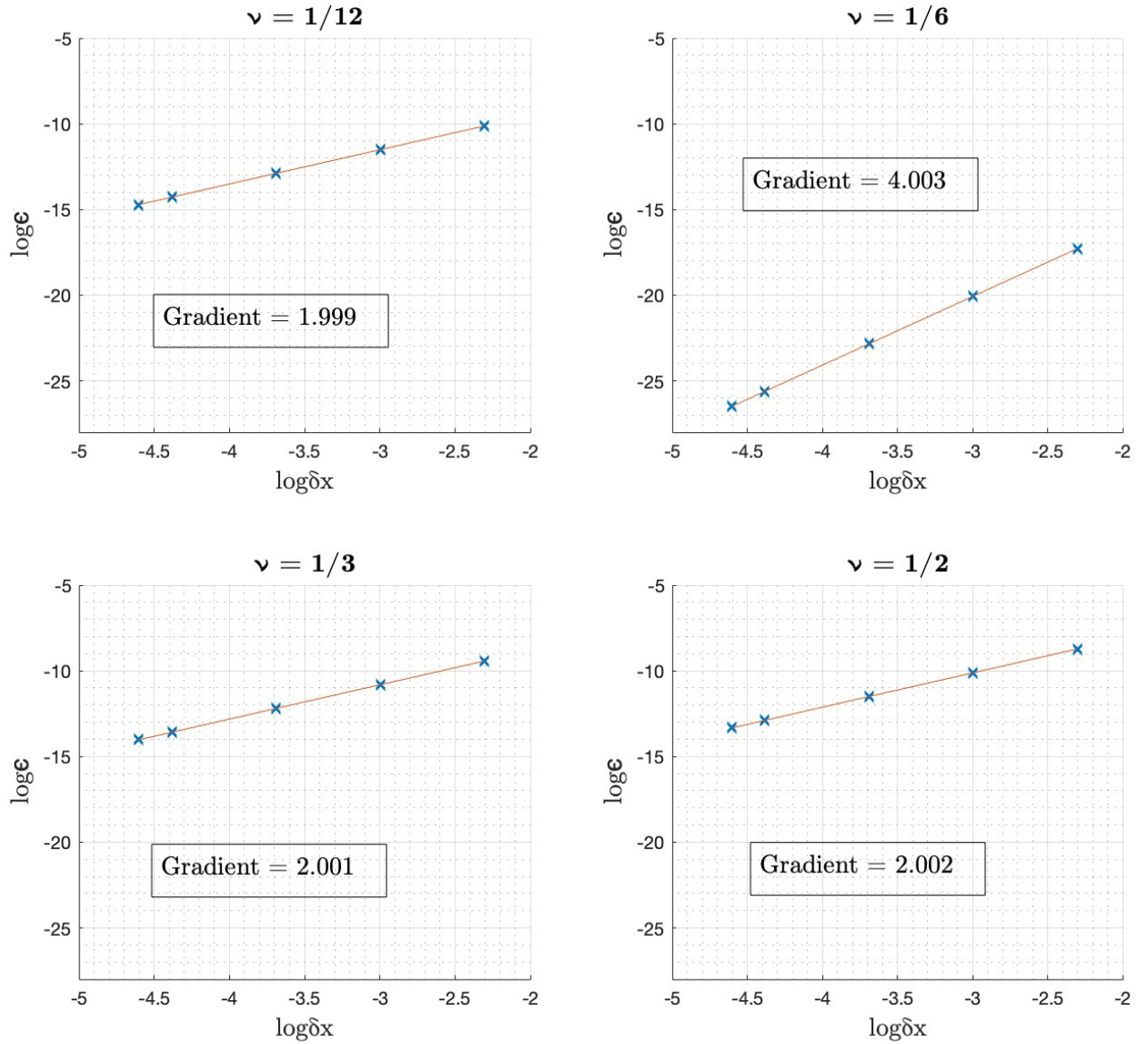
$$\Rightarrow \log \epsilon = 2 \log \delta x + c$$

Figure 2.6 illustrates four scatter plots of $\log \epsilon$ against $\log \delta x$ at $x = 0.5$ & $t = 0.5$ for $\nu = \frac{1}{12}, \frac{1}{6}, \frac{1}{3}$ & $\frac{1}{2}$ along with a line of best fit. Indeed, very similar results were obtained for other values of x & t and hence we feature only one representative figure here. For each value of ν , scheme (i) was used with $N = 10, 20, 40, 80$ & 100 . We find that for $\nu = \frac{1}{12}, \frac{1}{3}$ & $\frac{1}{2}$ the gradients of line of best fits are very nearly 2, in agreement with our above theoretical result for the order of scheme (i). However, for $\nu = \frac{1}{6}$ we find the gradient of the line of best fit to be very close to 4. This is in fact an interesting feature of the scheme: If $\nu = \frac{1}{6}$, the local truncation error of scheme (i) becomes

$$\eta = \mathcal{O}(\delta t^2) + \mathcal{O}(\delta x^4)$$

(Smith, G.D. pp. 40) resulting from the cancellation of lower order terms.

Figure 2.6 $\log \epsilon$ against $\log \delta x$ using scheme (i) for $\nu = \frac{1}{12}, \frac{1}{6}, \frac{1}{3}$ & $\frac{1}{2}$



Let us now look at scheme (iii). This scheme is unconditionally stable and convergent for $\frac{1}{2} \leq \rho \leq 1$ however for $0 \leq \rho < \frac{1}{2}$ we require that

$$\nu = \frac{\delta t}{\delta x^2} \leq \frac{1}{2(1 - 2\rho)}$$

for stability and convergence (Smith, G.D pp. 29).

Taking $\delta t = \nu \delta x^2$, the local truncation error of scheme (iii) will in general be

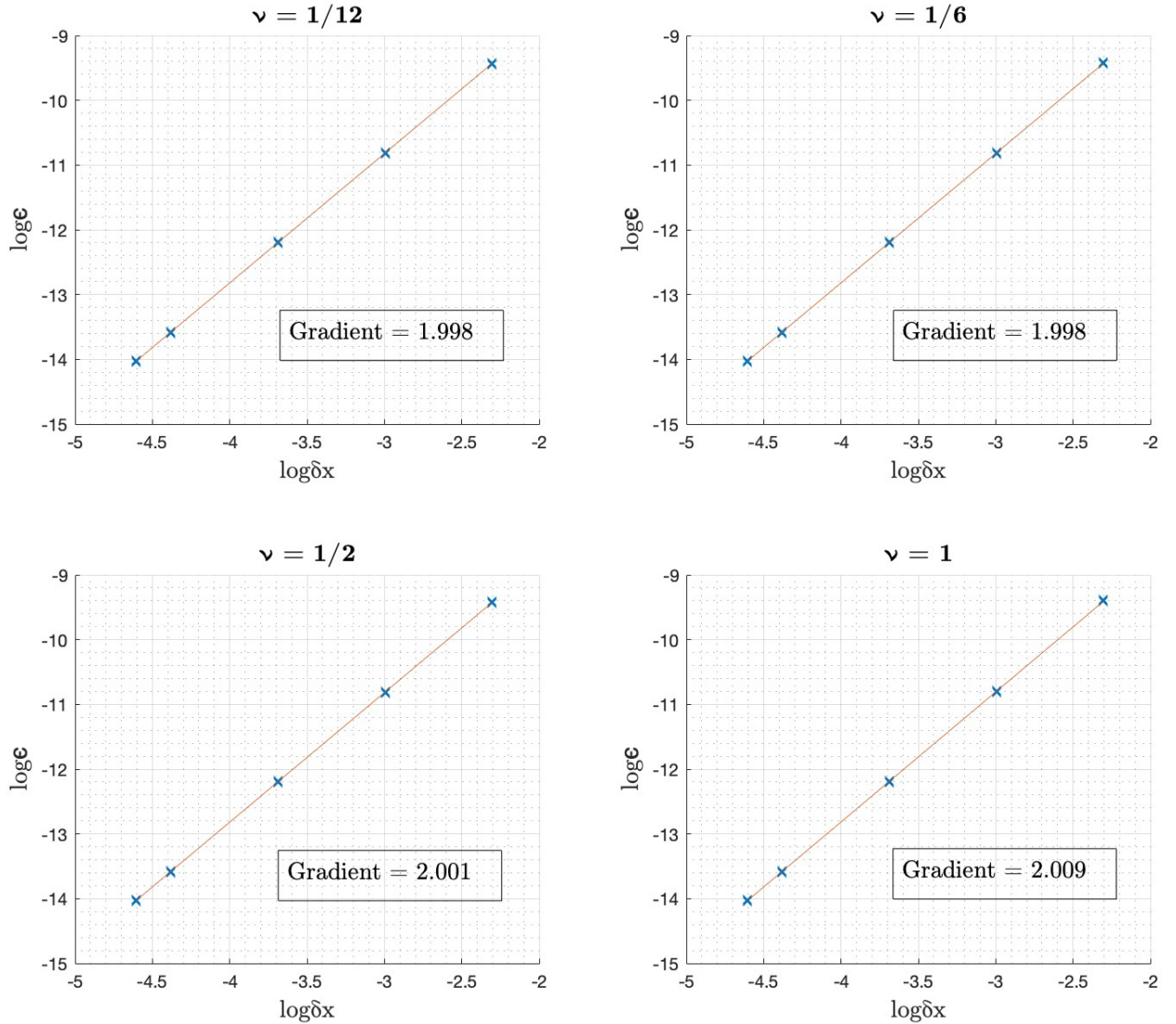
$$\eta = \mathcal{O}(\delta x^2)$$

(Crandall, S.H.). However, if

$$\rho = \frac{1}{2} \left(1 - \frac{1}{6\nu} \right) \quad (*)$$

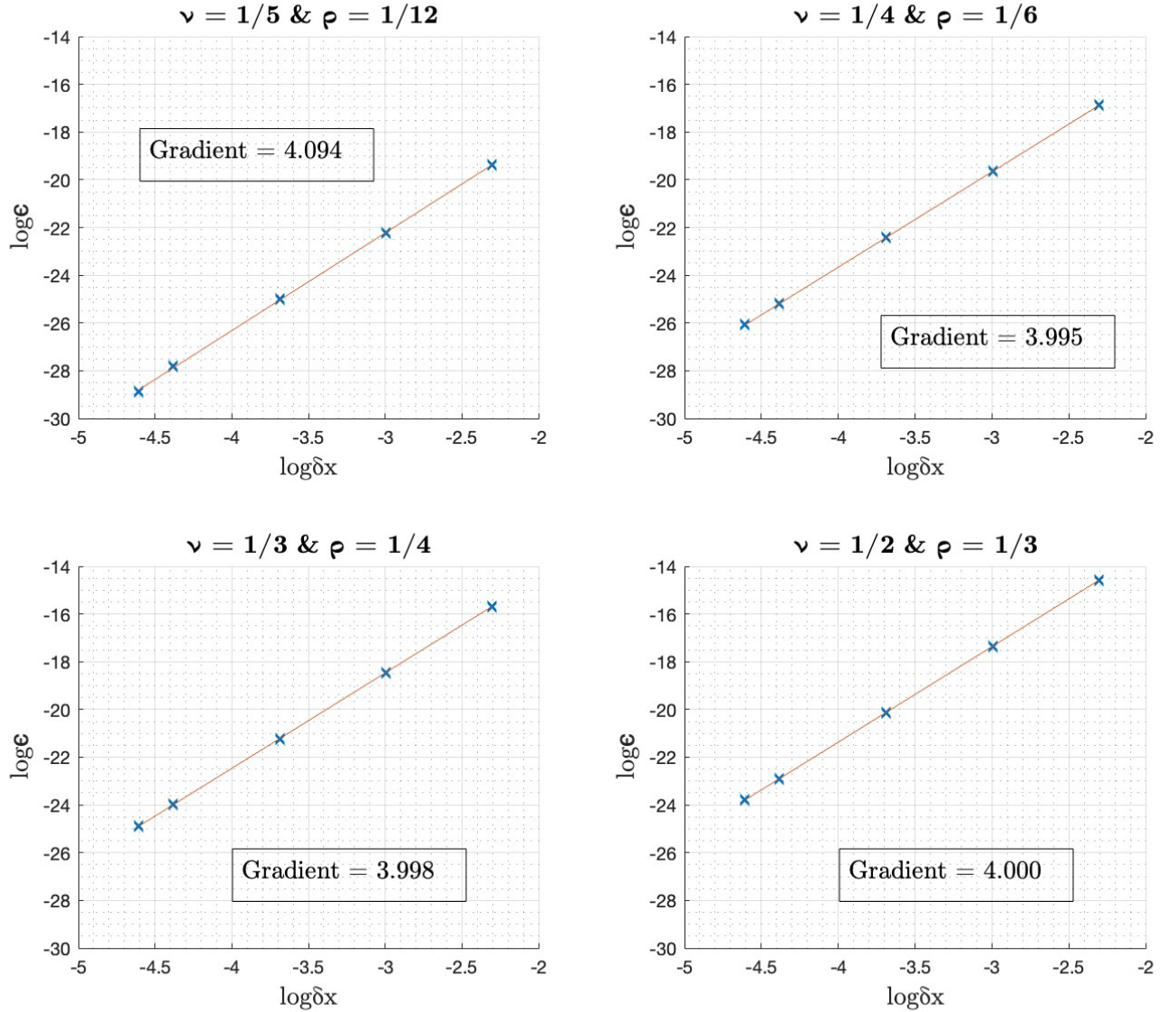
then the local truncation error becomes $\eta = \mathcal{O}(\delta x^4)$ (Crandall, S.H.). This is a generalisation of what we saw with scheme (i) when $\nu = \frac{1}{6}$ and indeed $\rho = 0$ & $\nu = \frac{1}{6}$ satisfy (*).

Figure 2.7 $\log \epsilon$ against $\log \delta x$ using scheme (iii) with $\rho = \frac{1}{2}$ for $\nu = \frac{1}{12}, \frac{1}{6}, \frac{1}{2}$ & 1



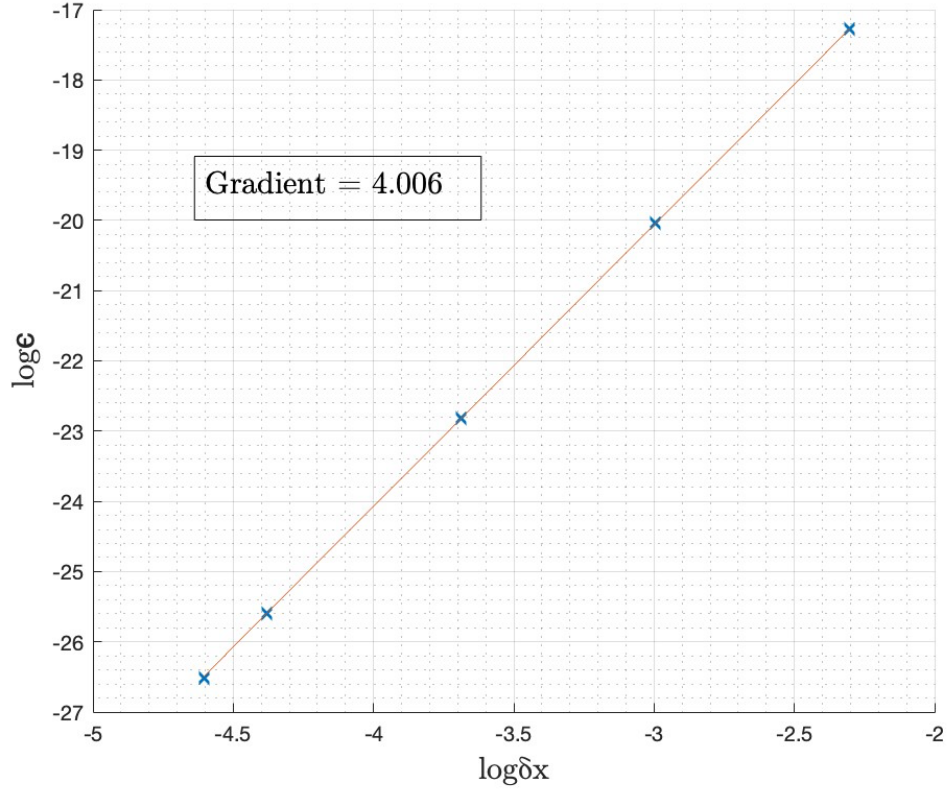
Figures 2.7 & 2.8 illustrate four scatter plots of $\log \epsilon$ against $\log \delta x$ at $x = 0.5$ & $t = 0.5$ for various ν & ρ as well as a line of best fit. Again, only $x = 0.5$ & $t = 0.5$ is featured here as a representative example. In each plot, scheme (iii) was used with $N = 10, 20, 40, 80$ & 100 . In Figure 2.7 we see that in each plot the gradient of the line of best fit is very nearly 2, in agreement with the theoretical result for the order of scheme (iii) for general ν & ρ . Figure 2.8 features selected pairs of values of ν & ρ which satisfy (*) and indeed, in each plot the gradient of the line of best fit is very nearly 4, in agreement with Crandall's result. We notice that out of the four plots $\nu = \frac{1}{5}$ & $\rho = \frac{1}{12}$ produced the smallest errors.

Figure 2.8 $\log \epsilon$ against $\log \delta x$ using scheme (iii) for selected ν & ρ



Crandall also identifies the pair of parameter values $\nu = \frac{\sqrt{5}}{10}$ & $\rho = \frac{3-\sqrt{5}}{6}$ for which the fourth order terms cancel and hence the local truncation error becomes $\eta = \mathcal{O}(\delta x^6)$. However, as illustrated in Figure 2.9, in practise it was only possible to achieve errors of order $\mathcal{O}(\delta x^4)$ using these parameter values.

Figure 2.9 $\log \epsilon$ against $\log \delta x$ using scheme (iii) for $v = \frac{\sqrt{5}}{10}$ & $\rho = \frac{3-\sqrt{5}}{6}$ at $x = 0.5$ & $t = \frac{1}{\sqrt{5}}$

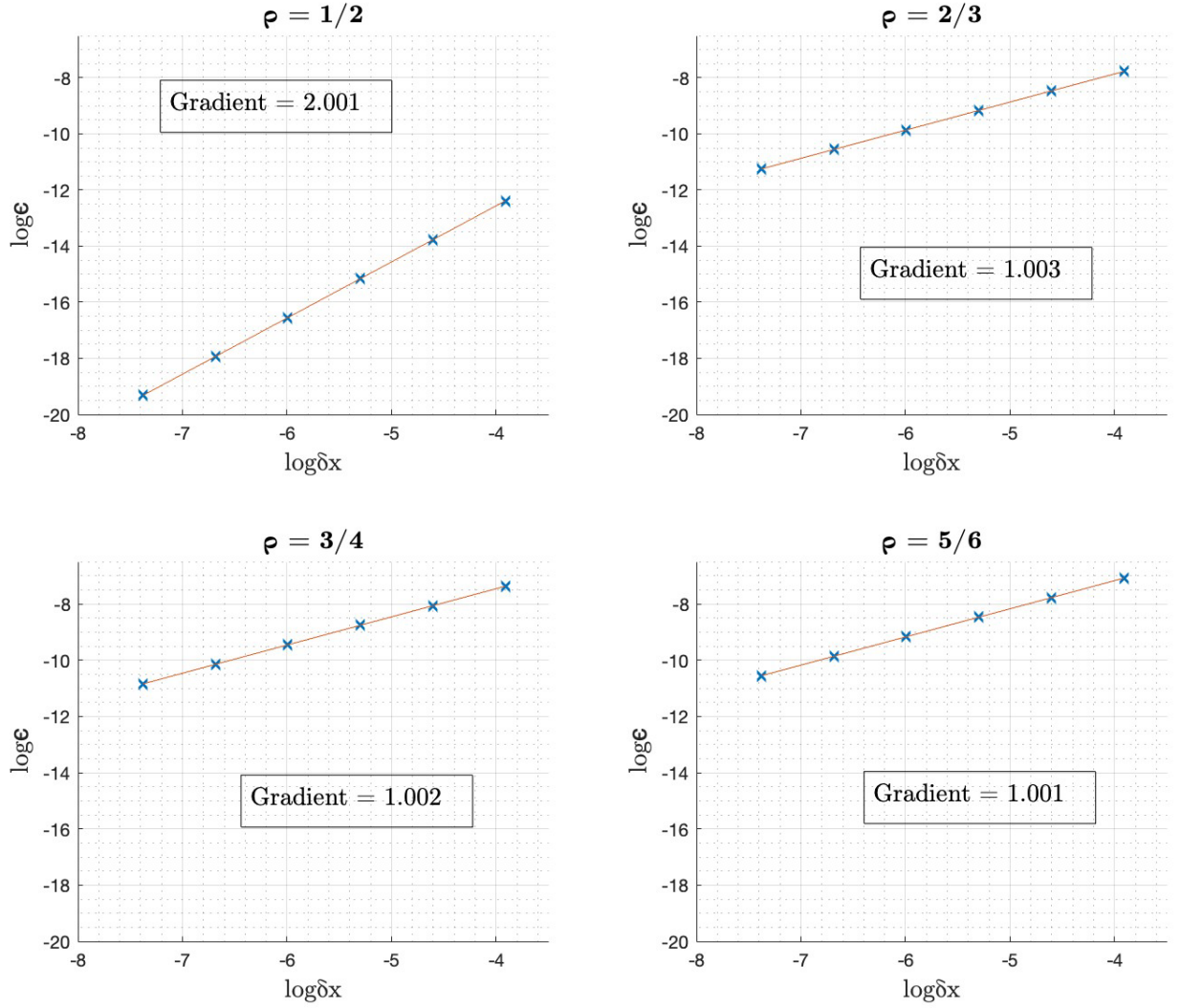


Consider now taking $\delta t = \mu \delta x$ as opposed to $\delta t = v \delta x^2$. The advantage of this is that now scheme (iii) marches the solution through time in a total of $\mathcal{O}(N)$ steps rather than $\mathcal{O}(N^2)$ steps. Hence, we can implement scheme (iii) with greater values of N at a reduced computational expense. Figure 2.10 illustrates four scatter plots of $\log \epsilon$ against $\log \delta x$ at $x = 0.5$ & $t = 2$ with a line of best fit. This time, in each plot, scheme (iii) was used with $\delta t = \delta x$ and $N = 50, 100, 200, 400, 800$ & 1600 . Using the stability condition for scheme (iii), if we take $\delta t = \mu \delta x$ then we must have $\rho \geq \frac{1}{2}$ to maintain stability as N increases. We find that for general $\rho > \frac{1}{2}$, the local truncation error is $\eta = \mathcal{O}(\delta x)$, however for $\rho = \frac{1}{2}$ the local truncation error is $\eta = \mathcal{O}(\delta x^2)$, independent of the value of μ . Indeed, explicitly, the truncation error of scheme (iii) is

$$\eta = \left(\left(\frac{1}{2} - \rho \right) \delta t - \frac{1}{12} \delta x^2 \right) \theta_{tt} + \dots$$

and hence when $\rho = \frac{1}{2}$, the δt term cancels and we have $\eta = \mathcal{O}(\delta x^2)$.

Figure 2.10 $\log \epsilon$ against $\log \delta x$ using scheme (iii) with $\delta t = \delta x$ for selected ρ



(iv) From our investigation thus far, we suggest the following three possible pairs of numerical schemes and parameter values which might be recommended to achieve a given level of accuracy using the least computing resources:

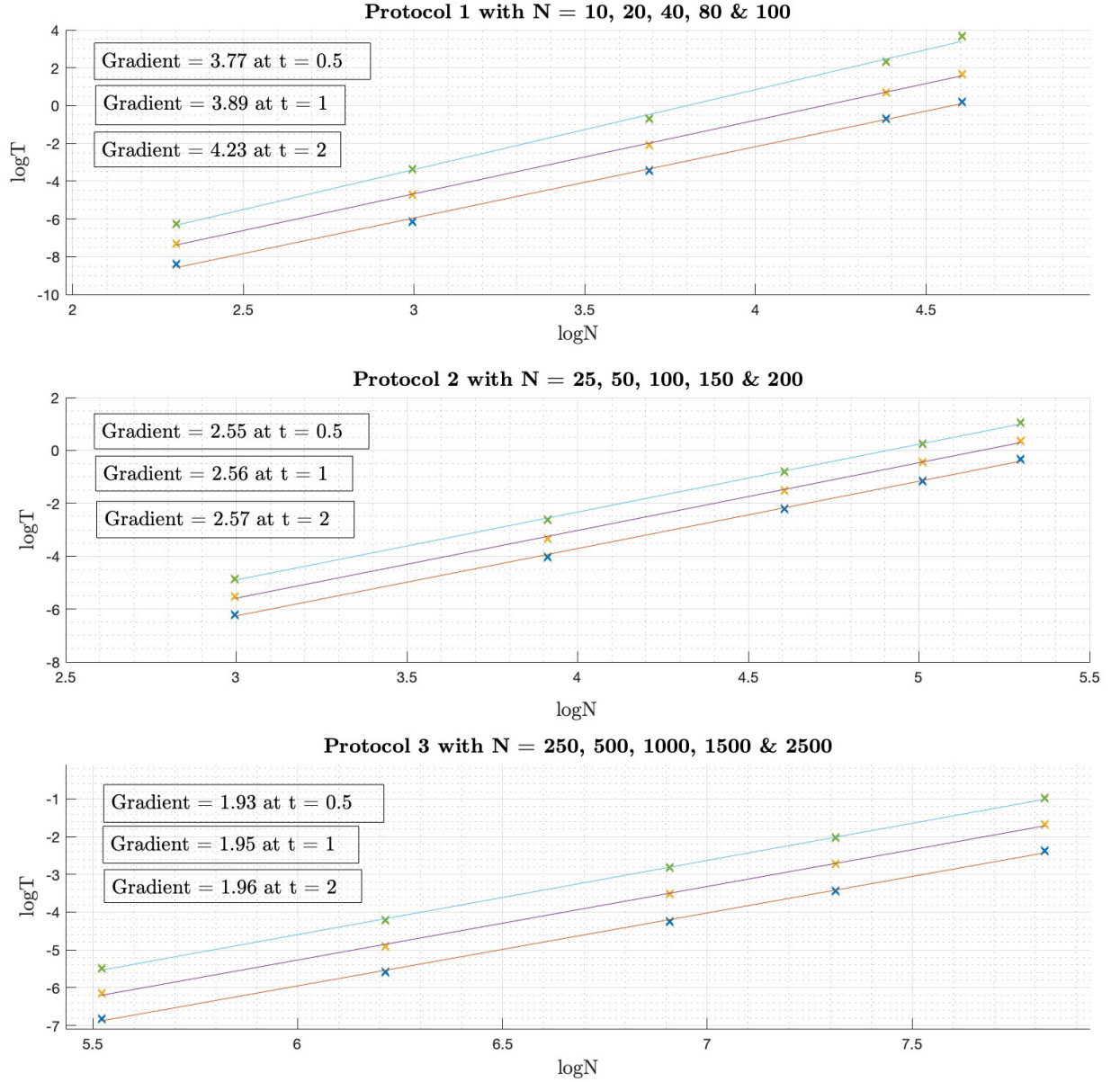
1. Scheme (i) with $\nu = \frac{1}{6}$
2. Scheme (iii) with $\nu = \frac{1}{5}$ & $\rho = \frac{1}{12}$
3. Scheme (iii) with $\delta t = \delta x$ & $\rho = \frac{1}{2}$

For each of these protocols for solving θ numerically our previous investigations found the errors to behave as:

1. $\epsilon \propto N^{-4}$
2. $\epsilon \propto N^{-4}$
3. $\epsilon \propto N^{-2}$

Protocols 1 & 2 are suggested for their reduced error, being 4th order as opposed to 2nd order, and hence offer greater accuracy for smaller N . Protocol 3 is suggested for its reduced operational complexity which, as discussed previously, allows for a larger value of N to be used in evaluation (in the hope of greater accuracy) at reduced computational cost.

Figure 2.11 $\log T$ against $\log N$ for each protocol evaluated at $t = 0.5, 1$ & 2



The operational complexity of each protocol was investigated by looking at the relationship between $\log T$ and $\log N$ with the intention of using the computational time taken T as a measure of the total operation count (assuming it takes our computer fixed duration of time to perform each elementary operation). Figure 2.11 illustrates this relationship for each protocol for $t = 0.5, 1$ & 2 . In each case the relationship is linear and hence for each protocol we find that the

computational time is polynomial in N . If we look, for example, at $t = 1$ we have the (approximate) relationships:

1. $T \propto N^{3.9} \Rightarrow T \propto \epsilon^{-0.975}$
2. $T \propto N^{2.6} \Rightarrow T \propto \epsilon^{-0.65}$
3. $T \propto N^{2.0} \Rightarrow T \propto \epsilon^{-1.0}$

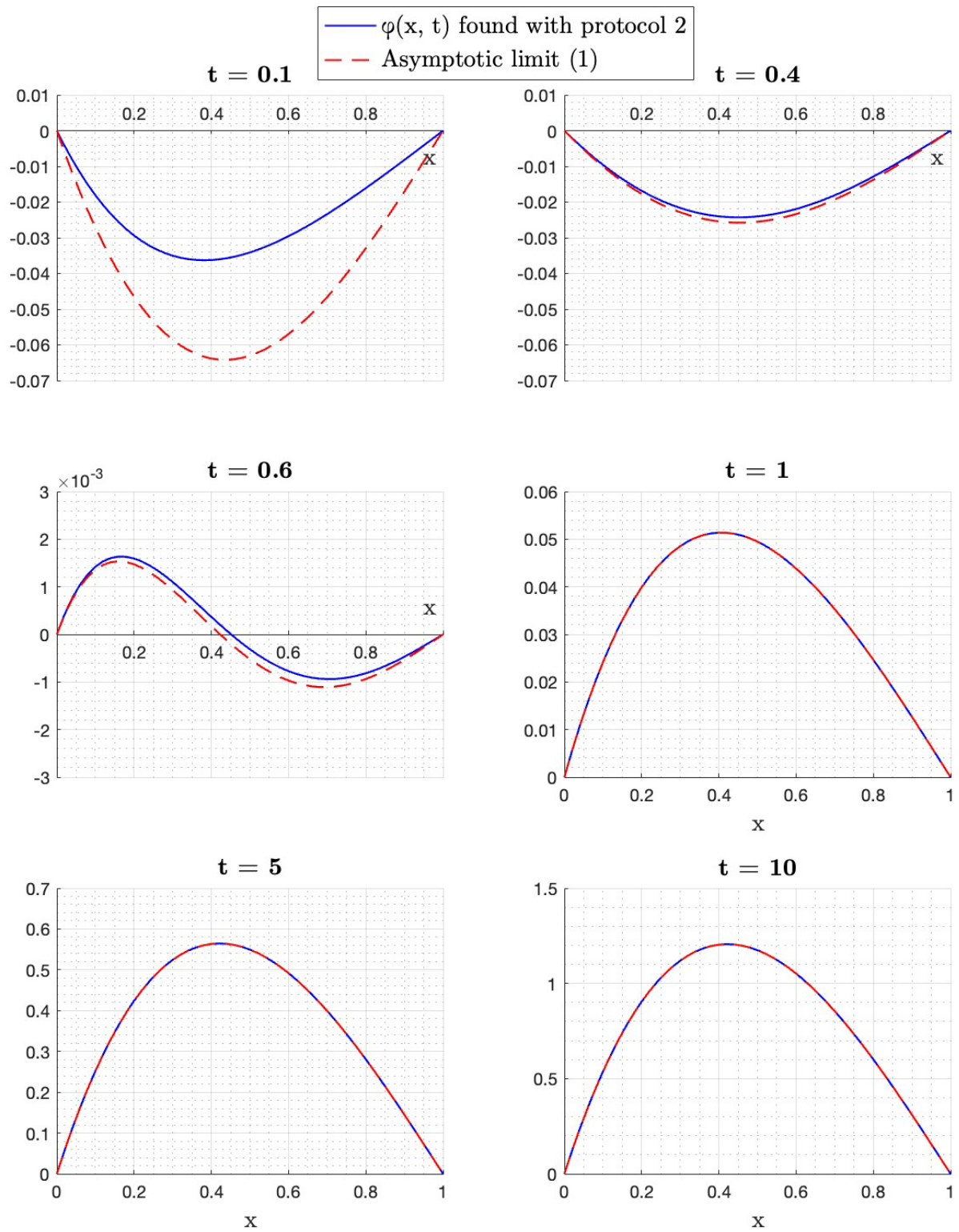
This indicates that out of the three suggested protocols, it is protocol 2 that requires the smallest number of operations to achieve a given level of accuracy and, indeed, this is also the case if we look at other values of t . Hence, we recommend using protocol 2 (scheme (iii) with $\nu = \frac{1}{5}$ & $\rho = \frac{1}{12}$) to achieve a given level of accuracy using the least computing resources.

(v) Figure 2.12 (overleaf) illustrates the convergence of $\phi(x, t)$ to the asymptotic limit (1) as t increases. Scheme (iii) was used with $N = 50$ to find θ and hence ϕ using

$$\phi(x, t) = \theta(x, t) - f(t)(1 - x).$$

The asymptotic limit $\alpha(x)t + \beta(x)$ was found by summing the first 500 terms of the series expressions for α & β .

Figure 2.12 $\phi(x, t)$ found numerically with protocol 2 and the asymptotic limit (1)



References

Listed in order of appearance

- [1] Ames, W.F. *Numerical Methods for Partial Differential Equations*, Academic Press.
- [2] Smith, G.D. *Numerical Solution of Partial Differential Equations: Finite Difference Methods*, OUP.
- [3] Iserles, A. *A First Course in the Numerical Analysis of Differential Equations*, CUP.
- [4] Crandall, S.H. *Quart. Appl. Math.* **13** (1955), 318-320, AMS.

Programs

All code written in MATLAB version R2024b

Question 1

Program computing the analytic solution by summing a finite number of terms

```
function theta_t = PPDEsQuestion1partiii(N,t)
% Sums the first N terms of the Fourier sine series for phi at given t
% INPUTS
% N - number of terms to be summed
% t - value of t phi is to be evaluated at
% OUTPUTS
% theta_t - function of x
syms n
theta_t = @(x) t*(1-t)*(1-x) + double(symsum(((4*t)/((n*pi)^3) -
(2/((n*pi)^3) + 4/((n*pi)^5))*(1-exp(-((n*pi)^2)*t))*sin(n*pi*x),n,1,N));
```

Program to produce Figure 1.1

```
N = 100;
syms n

alpha = @(x) double(symsum((4/((n*pi)^3))*sin(n*pi*x),n,1,N));
beta = @(x) (-1)*double(symsum((2/((n*pi)^3) +
4/((n*pi)^5))*sin(n*pi*x),n,1,N));

subplot(1,2,1)
fplot(alpha, [0 1], 'LineWidth', 1.2)
ylim([-0.01 0.15])
ax = gca;
ax.XAxisLocation = 'origin';
grid(ax,"on")
grid(ax,'minor')
xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 12)
ylabel('α(x)', 'FontName', 'CMU Serif', 'FontSize', 12)

subplot(1,2,2)
fplot(beta, [0 1], 'LineWidth', 1.2, 'Color', "#D95319")
ylim([-0.09 0.01])
ax = gca;
ax.XAxisLocation = 'origin';
```

```

grid(ax,"on")
grid(ax,'minor')
xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 12)
ylabel('β(x)', 'FontName', 'CMU Serif', 'FontSize', 12)

```

Program to produce Figure 1.2

```

time = [0.1 0.5 1 1.5 2 3];
ylow = [-0.1 -0.1 -0.1 -1 -2.25 -6.25];
yhi = [0.3 0.3 0.3 0.2 0.5 0.8];
for i = 1:length(time)
    subplot(3,2,i)
    theta = PPDEsQuestion1partiii(100,time(i));
    fplot(theta,[0 1], 'LineWidth',1.1)
    ylim([ylow(i) yhi(i)])
    ax = gca;
    ax.XAxisLocation = 'origin';
    grid(ax,"on")
    grid(ax,'minor')
    xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 12)
    ylabel('θ(x,t)', 'FontName', 'CMU Serif', 'FontSize', 12)
    str = "t = " + num2str(time(i));
    title(str, 'FontName', 'CMU Serif', 'FontSize', 12)
end

```

Question 2

Program implementing scheme (i)

```

function theta = PPDEsSchemei(N,t,nu)
% Solves for theta using scheme (i)
% INPUTS
% N
% t - value of t we wish evaluate theta up to
% nu - Courant number
% OUTPUTS
% theta - matrix of theta values where theta(n+1,m+1) approximates θ(nδx, mδt)

% Set up + initial conditions
deltax = 1/N;
deltat = nu*(deltax^2);
M = round(t/deltat);
theta = zeros(N+1,M+1);

for m = 1:M
    for n = 2:N
        theta(n,m+1) = theta(n,m) + nu*(theta(n+1,m)-2*theta(n,m)+theta(n-1,m));
    end
    % Boundary conditions
    theta(1,m+1) = ((m)*deltat)*(1-((m)*deltat));
    theta(N+1,:) = 0;
end

```

Program implementing scheme (ii)

```
function theta = PPDEsSchemeii(N,t,nu)
% Solves for theta using scheme (ii)
% INPUTS
% N
% t - value of t we wish evaluate theta up to
% nu - Courant number
% OUTPUTS
% theta - matrix of theta values where theta(n+1,m+1) approximates  $\theta(n\delta x, m\delta t)$ 

% Set up + initial conditions
deltax = 1/N;
deltat = nu*(deltax^2);
M = round(t/deltat);
theta = zeros(N+1,M+1);

% Using (i) for first step:
for n = 2:N
    theta(n,2) = theta(n,1) + nu*(theta(n+1,1)-2*theta(n,1)+theta(n-1,1));
end
theta(1,2) = (deltat)*(1-(deltat));
theta(N+1,2) = 0;

% Implementing (ii) for remaining steps
for m = 2:M
    for n = 2:N
        theta(n,m+1) = theta(n,m-1) + 2*nu*(theta(n+1,m)-
        2*theta(n,m)+theta(n-1,m));
    end
    % Boundary conditions
    theta(1,m+1) = ((m)*deltat)*(1-((m)*deltat));
    theta(N+1,m+1) = 0;
end
```

Program implementing scheme (iii)

```
function theta = PPDEsSchemeiii(N,t,nu,rho)
% Solves for theta using scheme (iii)
% INPUTS
% N
% t - value of t we wish evaluate theta up to
% nu - Courant number
% rho - parameter
% OUTPUTS
% theta - matrix of theta values where theta(n+1,m+1) approximates  $\theta(n\delta x, m\delta t)$ 

% Set up + initial conditions
deltax = 1/N;
deltat = nu*(deltax^2);
M = round(t/deltat);
theta = zeros(N+1,M+1);
SpM1 = spdiags([-rho*nu 1+2*rho*nu -rho*nu], -1:1,N-1,N-1);
SpM2 = spdiags([(1-rho)*nu 1-2*(1-rho)*nu (1-rho)*nu], -1:1,N-1,N-1);

for m = 1:M
```

```

    Rhs = SpM2*theta(2:N,m);
    Rhs(1) = Rhs(1) + rho*nu*((m)*deltat)*(1-((m)*deltat)) + (1-
rho)*nu*((m-1)*deltat)*(1-((m-1)*deltat));
    theta(2:N,m+1) = SpM1\Rhs;
    % Boundary conditions
    theta(1,m+1) = ((m)*deltat)*(1-((m)*deltat));
    theta(N+1,m+1) = 0;
end

```

Program used to produce Figures 2.1 – 2.3

```

N = 5;
nu = 1/2;
rho = 1/2;

deltax = 1/N;
deltat = nu*(deltax^2);

t = [0.1 0.5 1 1.5];
ylow = [-0.1 -0.1 -0.1 -1];
yhi = [0.3 0.3 0.3 0.2];

f1 = figure;
thetai = PPDEsSchemei(N,t(end),nu);
for i = 1:length(t)
    subplot(2,2,i)
    hold on
    plot(0:deltax:1,thetai(:,round(t(i)/deltat)+1), 'LineWidth',1.1)
    ylim([ylow(i) yhi(i)])
    ax = gca;
    ax.XAxisLocation = 'origin';
    grid(ax,"on")
    grid(ax,'minor')
    xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 12)
    ylabel('θ(x,t)', 'FontName', 'CMU Serif', 'FontSize', 12)
    str = "t = " + num2str(t(i));
    title(str, 'FontName', 'CMU Serif', 'FontSize', 12)
end

f2 = figure;
thetaiii = PPDEsSchemeiii(N,t(end),nu,rho);
for i = 1:length(t)
    subplot(2,2,i)
    hold on
    plot(0:deltax:1,thetaiii(:,round(t(i)/deltat)+1), 'LineWidth',1.1)
    ylim([ylow(i) yhi(i)])
    ax = gca;
    ax.XAxisLocation = 'origin';
    grid(ax,"on")
    grid(ax,'minor')
    xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 12)
    ylabel('θ(x,t)', 'FontName', 'CMU Serif', 'FontSize', 12)
    str = "t = " + num2str(t(i));
    title(str, 'FontName', 'CMU Serif', 'FontSize', 12)
end

f3 = figure;
thetaii = PPDEsSchemeii(N,t(end),nu);
for i = 1:2
    subplot(1,2,i)

```

```

hold on
plot(0:deltax:1,thetai(:,round(t(i)/deltat)+1), 'LineWidth',1.1)
ax = gca;
ax.XAxisLocation = 'origin';
grid(ax,"on")
grid(ax,'minor')
xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 12)
ylabel('θ(x,t)', 'FontName', 'CMU Serif', 'FontSize', 12)
str = "t = " + num2str(t(i));
title(str, 'FontName', 'CMU Serif', 'FontSize', 12)
end

```

Program used to produce Table 2.1

```

format long
N = 5;
nu = 1/2;
rho = 1/2;
deltax = 1/N;
t = 0.1;

thetai = PPDEsSchemei(N,t,nu);
thetaiii = PPDEsSchemeiii(N,t,nu,rho);
theta = PPDEsQuestion1partiii(20000,t);

x = transpose(0:deltax:1);
analytic = theta(x);
scheme_i = thetai(:,end);
scheme_iii = thetaiii(:,end);
error_i = abs(analytic - scheme_i);
error_iii = abs(analytic - scheme_iii);

table(x,analytic,scheme_i,scheme_iii,error_i,error_iii)

```

Program used to produce Figure 2.4

```

N = 5;
nu = 1/2;
rho = 1/2;
deltax = 1/N;
t = 0.1;

thetai = PPDEsSchemei(N,t,nu);
thetaiii = PPDEsSchemeiii(N,t,nu,rho);
theta = PPDEsQuestion1partiii(100,t);

subplot(1,2,1)
hold on
fplot(theta, [0 1], 'Color', 'k', 'LineWidth', 1.1)
plot(0:deltax:1,thetai(:,end), 'Color', 'magenta', 'LineWidth', 1.1)
ax = gca;
ax.XAxisLocation = 'origin';
grid(ax,"on")
grid(ax,'minor')
xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 12)
ylabel('θ(x,t)', 'FontName', 'CMU Serif', 'FontSize', 12)
legend('Analytic solution', 'Scheme (i)', 'FontName', 'CMU Serif',
'FontSize', 12)

```

```

subplot(1,2,2)
hold on
fplot(theta, [0 1], 'Color', 'k', 'LineWidth', 1.1)
plot(0:deltax:1,thetaiii(:,end), 'Color', 'r', 'LineWidth', 1.1)
ax = gca;
ax.XAxisLocation = 'origin';
grid(ax,"on")
grid(ax,'minor')
xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 12)
ylabel('θ(x,t)', 'FontName', 'CMU Serif', 'FontSize', 12)
legend('Analytic solution', 'Scheme (iii)', 'FontName', 'CMU Serif',
'FontSize', 12)

```

Program used to produce Figure 2.5

```

N = 5;
nu = 1/2;
rho = 1/2;
deltax = 1/N;
t = 0.1;

thetai = PPDEsSchemei(N,t,nu);
thetaiii = PPDEsSchemeiii(N,t,nu,rho);
theta = PPDEsQuestion1partiii(20000,t);
analytic = theta(transpose(0:deltax:1));
erri = abs(thetai(:,end) - analytic);
erriii = abs(thetaiii(:,end) - analytic);

hold on
plot(0:deltax:1,erri, 'LineWidth', 1.1)
plot(0:deltax:1,erriii, 'LineWidth', 1.1)
ax = gca;
ax.XAxisLocation = 'origin';
grid(ax,"on")
grid(ax,'minor')
xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 12)
ylabel('Error', 'FontName', 'CMU Serif', 'FontSize', 12)
legend('Error of scheme (i)', 'Error of scheme (iii)', 'FontName', 'CMU
Serif', 'FontSize', 12)

```

Program used to produce Figure 2.6

```

N = [10 20 40 80 100];
nu = [1/12 1/6 1/3 1/2];
nu2 = ["ν = 1/12", "ν = 1/6", "ν = 1/3", "ν = 1/2"];

x = 0.5;
t = 0.5;
theta = PPDEsQuestion1partiii(20000,t);
analytic = theta(x);
erri = zeros(length(N),length(nu));
for j = 1:length(nu)
    for k = 1:length(N)
        deltax = 1/N(k);
        thetai = PPDEsSchemei(N(k),t,nu(j));
        erri(k,j) = abs(analytic - thetai(round(x/deltax)+1,end));
    end
end

```

```

subplot(2,2,j)
hold on
scatter(log(1./N),log(erri(:,j)), 'x', 'LineWidth', 2)
ax = gca;
grid(ax,"on")
grid(ax,'minor')
xlabel('log $\delta x$ ', 'FontName', 'CMU Serif', 'FontSize', 14)
ylabel('log $\epsilon$ ', 'FontName', 'CMU Serif', 'FontSize', 14)
xlim([-5 -2])
ylim([-28 -5])
lob = polyfit(log(1./N),log(erri(:,j)),1)
plot(log(1./N),polyval(lob,log(1./N)))
str = "Gradient = " + num2str(round(lob(1),3));
annotation('textbox', 'String', str, 'FontName', 'CMU Serif',
'FontSize', 14)
title(nu2(j), 'FontName', 'CMU Serif', 'FontSize', 14)
end

```

Program used to produce Figure 2.7

```

N = [10 20 40 80 100];
nu = [1/12 1/6 1/2 1];
nu2 = [" $\nu = 1/12$ ", " $\nu = 1/6$ ", " $\nu = 1/2$ ", " $\nu = 1$ "];
rho = 1/2;

x = 0.5;
t = 0.5;
theta = PPDEsQuestion1partiii(20000,t);
analytic = theta(x);
erriii = zeros(length(N),length(nu));
for j = 1:length(nu)
    for k = 1:length(N)
        deltax = 1/N(k);
        thetaiii = PPDEsSchemeiii(N(k),t,nu(j),rho);
        erriii(k,j) = abs(analytic - thetaiii(round(x/deltax)+1,end));
    end
    subplot(2,2,j)
    hold on
    scatter(log(1./N),log(erriii(:,j)), 'x', 'LineWidth', 2)
    ax = gca;
    grid(ax,"on")
    grid(ax,'minor')
    xlabel('log $\delta x$ ', 'FontName', 'CMU Serif', 'FontSize', 14)
    ylabel('log $\epsilon$ ', 'FontName', 'CMU Serif', 'FontSize', 14)
    ylim([-15 -9])
    lob = polyfit(log(1./N),log(erriii(:,j)),1)
    plot(log(1./N),polyval(lob,log(1./N)))
    str = "Gradient = " + num2str(round(lob(1),3));
    annotation('textbox', 'String', str, 'FontName', 'CMU Serif',
'FontSize', 14)
    title(nu2(j), 'FontName', 'CMU Serif', 'FontSize', 14)
end

```

Program used to produce Figure 2.8

```

N = [10 20 40 80 100];
nu = [1/5 1/4 1/3 1/2];
rho = [1/12 1/6 1/4 1/3];

```

```

titles = ["v = 1/5 & rho = 1/12", "v = 1/4 & rho = 1/6", "v = 1/3 & rho = 1/4", "v = 1/2 & rho = 1/3"];

x = 0.5;
t = 0.5;
theta = PPDEsQuestion1partiii(20000,t);
analytic = theta(x);
erriii = zeros(length(N),length(nu));
for j = 1:length(nu)
    for k = 1:length(N)
        deltax = 1/N(k);
        thetaiii = PPDEsSchemeiii(N(k),t,nu(j),rho(j));
        erriii(k,j) = abs(analytic - thetaiii(round(x/deltax)+1,end));
    end
    subplot(2,2,j)
    hold on
    scatter(log(1./N),log(erriii(:,j)), 'x', 'LineWidth', 2)
    ax = gca;
    grid(ax,"on")
    grid(ax,'minor')
    xlabel('logδx', 'FontName', 'CMU Serif', 'FontSize', 14)
    ylabel('loge', 'FontName', 'CMU Serif', 'FontSize', 14)
    ylim([-30 -14])
    lob = polyfit(log(1./N),log(erriii(:,j)),1)
    plot(log(1./N),polyval(lob,log(1./N)))
    str = "Gradient = " + num2str(round(lob(1),3));
    annotation('textbox', 'String', str, 'FontName', 'CMU Serif',
'FontSize', 14)
    title(titles(j), 'FontName', 'CMU Serif', 'FontSize', 14)
end

```

Program used to produce Figure 2.9

```

N = [10 20 40 80 100];
nu = sqrt(5)/10;
rho = (3-sqrt(5))/6;

x = 0.5;
t = 1/sqrt(5);
theta = PPDEsQuestion1partiii(20000,t);
analytic = theta(x);
erriii = zeros(1,length(N));
for k = 1:length(N)
    deltax = 1/N(k);
    thetaiii = PPDEsSchemeiii(N(k),t,nu,rho);
    erriii(k) = abs(analytic - thetaiii(round(x/deltax)+1,end));
end
hold on
scatter(log(1./N),log(erriii), 'x', 'LineWidth', 2)
ax = gca;
grid(ax,"on")
grid(ax,'minor')
xlabel('logδx', 'FontName', 'CMU Serif', 'FontSize', 15)
ylabel('loge', 'FontName', 'CMU Serif', 'FontSize', 15)
lob = polyfit(log(1./N),log(erriii),1)
plot(log(1./N),polyval(lob,log(1./N)))
str = "Gradient = " + num2str(round(lob(1),3));
annotation('textbox', 'String', str, 'FontName', 'CMU Serif', 'FontSize',
15)

```

Program used to produce Figure 2.10

```
N = [50 100 200 400 800 1600];
mu = 1;
mu2 = ["p = 1/2", "p = 2/3", "p = 3/4", "p = 5/6"];
rho = [1/2 2/3 3/4 5/6];

x = 0.5;
t = 2;
theta = PPDEsQuestion1partiii(20000,t);
analytic = theta(x);
errii = zeros(length(N),length(rho));
for j = 1:length(rho)
    for k = 1:length(N)
        deltax = 1/N(k);
        thetaii = PPDEsSchemeiii(N(k),t,mu*N(k),rho(j));
        errii(k,j) = abs(analytic - thetaii(round(x/deltax)+1,end));
    end
    subplot(2,2,j)
    hold on
    scatter(log(1./N),log(errii(:,j)), 'x', 'LineWidth', 2)
    ax = gca;
    grid(ax,"on")
    grid(ax,'minor')
    xlabel('logδx', 'FontName', 'CMU Serif', 'FontSize', 14)
    ylabel('loge', 'FontName', 'CMU Serif', 'FontSize', 14)
    xlim([-8 -3.5])
    ylim([-20 -6.5])
    lob = polyfit(log(1./N),log(errii(:,j)),1)
    plot(log(1./N),polyval(lob,log(1./N)))
    str = "Gradient = " + num2str(round(lob(1),3));
    annotation('textbox', 'String', str, 'FontName', 'CMU Serif',
'FontSize', 14)
    title(mu2(j), 'FontName', 'CMU Serif', 'FontSize', 14)
end
```

Program used to produce Figure 2.11

```
t = [0.5 1 2];

N1 = [10 20 40 80 100];
nu1 = 1/6;

time1 = [];

subplot(3,1,1)
hold on
for j = 1:length(t)
    f1 = @() PPDEsSchemei(N1(1),t(j),nu1);
    time1(1) = timeit(f1);

    f2 = @() PPDEsSchemei(N1(2),t(j),nu1);
    time1(2) = timeit(f2);

    f3 = @() PPDEsSchemei(N1(3),t(j),nu1);
    time1(3) = timeit(f3);
```

```

f4 = @() PPDEsSchemei(N1(4),t(j),nu1);
time1(4) = timeit(f4);

f5 = @() PPDEsSchemei(N1(5),t(j),nu1);
time1(5) = timeit(f5);

scatter(log(N1),log(time1), 'x', 'LineWidth', 2)
ax = gca;
grid(ax,"on")
grid(ax,'minor')
xlabel('logN', 'FontName', 'CMU Serif', 'FontSize', 14)
ylabel('logT', 'FontName', 'CMU Serif', 'FontSize', 14)
lob = polyfit(log(N1),log(time1),1)
plot(log(N1),polyval(lob,log(N1)))
str1 = "Gradient = " + num2str(round(lob(1),3)) + " at t = " +
num2str(t(j));
annotation('textbox', 'String',str1, 'FontName', 'CMU Serif',
'FontSize', 14)
title("Protocol 1 with N = 10, 20, 40, 80 & 100", 'FontName', 'CMU
Serif', 'FontSize', 14)
end

N2 = [20 50 100 150 200];
nu2 = 1/5;
rho2 = 1/12;

time2 = [];

subplot(3,1,2)
hold on
for j = 1:length(t)
    f1 = @() PPDEsSchemeiii(N2(1),t(j),nu2,rho2);
    time2(1) = timeit(f1);

    f2 = @() PPDEsSchemeiii(N2(2),t(j),nu2,rho2);
    time2(2) = timeit(f2);

    f3 = @() PPDEsSchemeiii(N2(3),t(j),nu2,rho2);
    time2(3) = timeit(f3);

    f4 = @() PPDEsSchemeiii(N2(4),t(j),nu2,rho2);
    time2(4) = timeit(f4);

    f5 = @() PPDEsSchemeiii(N2(5),t(j),nu2,rho2);
    time2(5) = timeit(f5);

    scatter(log(N2),log(time2), 'x', 'LineWidth', 2)
    ax = gca;
    grid(ax,"on")
    grid(ax,'minor')
    xlabel('logN', 'FontName', 'CMU Serif', 'FontSize', 14)
    ylabel('logT', 'FontName', 'CMU Serif', 'FontSize', 14)
    lob = polyfit(log(N2),log(time2),1)
    plot(log(N2),polyval(lob,log(N2)))
    str2 = "Gradient = " + num2str(round(lob(1),3)) + " at t = " +
num2str(t(j));
    annotation('textbox', 'String',str2, 'FontName', 'CMU Serif',
'FontSize', 14)

```

```

        title("Protocol 2 with N = 25, 50, 100, 150 & 200", 'FontName', 'CMU
Serif', 'FontSize', 14)
end
N3 = [250 500 1000 1500 2500];
mu = 1;
rho3 = 1/2;

time3 = [];

subplot(3,1,3)
hold on
for j = 1:length(t)
    f1 = @() PPDEsSchemeiii(N3(1),t(j),mu*N3(1),rho3);
    time3(1) = timeit(f1);

    f2 = @() PPDEsSchemeiii(N3(2),t(j),mu*N3(2),rho3);
    time3(2) = timeit(f2);

    f3 = @() PPDEsSchemeiii(N3(3),t(j),mu*N3(3),rho3);
    time3(3) = timeit(f3);

    f4 = @() PPDEsSchemeiii(N3(4),t(j),mu*N3(4),rho3);
    time3(4) = timeit(f4);

    f5 = @() PPDEsSchemeiii(N3(5),t(j),mu*N3(5),rho3);
    time3(5) = timeit(f5);

    scatter(log(N3),log(time3), 'x', 'LineWidth', 2)
    ax = gca;
    grid(ax,"on")
    grid(ax,'minor')
    xlabel('logN', 'FontName', 'CMU Serif', 'FontSize', 14)
    ylabel('logT', 'FontName', 'CMU Serif', 'FontSize', 14)
    lob = polyfit(log(N3),log(time3),1)
    plot(log(N3),polyval(lob,log(N3)))
    str3 = "Gradient = " + num2str(round(lob(1),3)) + " at t = " +
num2str(t(j));
    annotation('textbox', 'String',str3, 'FontName', 'CMU Serif',
'FontSize', 14)
    title("Protocol 3 with N = 250, 500, 1000, 1500 & 2500", 'FontName',
'CMU Serif', 'FontSize', 14)
end

```

Program used to produce Figure 2.12

```

N = 50;
deltat = (1/5)/(N^2);
x = transpose(0:1/N:1);
t = [0.1 0.4 0.6 1 5 10];
yup = [0.01 0.01 0.003 0.06 0.7 1.5];
ylo = [-0.07 -0.07 -0.003 0 0 0];
theta = PPDEsSchemeiii(N,t(end),1/5,1/12);

Num = 500;
syms n
alpha = @(x) double(symsum((4/((n*pi)^3))*sin(n*pi*x),n,1,Num));
beta = @(x) (-1)*double(symsum((2/((n*pi)^3) +
4/((n*pi)^5))*sin(n*pi*x),n,1,Num));

```

```

for j = 1:length(t)
    phi = theta(:,round(t(j)/deltat) + 1) - t(j)*(1-t(j))*(1-x);
    asym = t(j)*alpha(x) + beta(x);
    subplot(3,2,j)
    hold on
    plot(x,phi,'LineWidth',1.1,'Color','b')
    plot(x,asym,'LineStyle','--','LineWidth',1.1,'Color','r')
    ylim([ylo(j) yup(j)])
    ax = gca;
    ax.XAxisLocation = 'origin';
    grid(ax,"on")
    grid(ax,'minor')
    xlabel('x', 'FontName', 'CMU Serif', 'FontSize', 14)
    str = "t = " + num2str(t(j));
    title(str, 'FontName', 'CMU Serif', 'FontSize', 14)
end
legend("φ(x, t) found with protocol 2", "Asymptotic limit (1)",
'FontName', 'CMU Serif', 'FontSize', 14)

```